

|                                                                             |   |
|-----------------------------------------------------------------------------|---|
| 4. FICHEROS. CONCEPTO DE “PERSISTENCIA” DE LOS DATOS DE UNA APLICACIÓN..... | 1 |
| 5. CLASIFICACIÓN DE FICHEROS.....                                           | 2 |
| 5.1. Contenido.....                                                         | 2 |
| 5.1.1. Ficheros de texto.....                                               | 2 |
| 5.1.2. Ficheros binarios.....                                               | 2 |
| 5.2. Modo de acceso.....                                                    | 2 |
| 5.2.1. Ficheros secuenciales.....                                           | 2 |
| 5.2.2. Fichero de acceso directo.....                                       | 3 |
| 6. APERTURA Y CIERRE DE FICHEROS.....                                       | 3 |
| 6.1. Creación de ficheros.....                                              | 3 |
| 6.2. Creación de directorios.....                                           | 5 |
| 6.3. Eliminación de ficheros y directorios.....                             | 6 |
| 6.4. Renombrar o mover ficheros y directorios.....                          | 7 |
| 6.5. Leer contenido de directorios.....                                     | 8 |

## 4. FICHEROS. CONCEPTO DE “PERSISTENCIA” DE LOS DATOS DE UNA APLICACIÓN

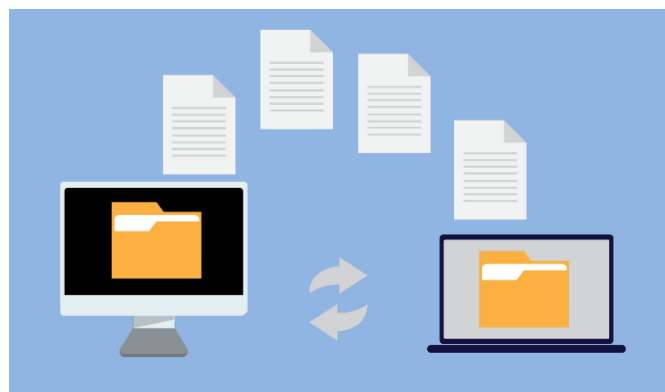


Almacenar la información en memoria tiene el problema de que al finalizar el programa toda esa información se pierde.

Sin embargo, gracias los ficheros podremos almacenar la información que necesitemos, de forma permanente, con la posibilidad de recuperarla más adelante. Al concepto de almacenar la información para luego poder recuperarla de nuevo se le denomina “persistencia de datos”, y podremos aplicarla tanto al contexto de ficheros como al de bases de datos.

Un **fichero** es una colección de datos organizados que se almacenan en un soporte externo de almacenamiento, como un disco duro, disco compacto, etc. Estos soportes no se borran cuando finaliza la ejecución del programa, ni siquiera cuando se apaga el ordenador.

Los ficheros no sólo nos serán útiles a la hora de almacenar la información, sino que también serán muy útiles a la hora de transferir información de un dispositivo a otro.



## 5. CLASIFICACIÓN DE FICHEROS

Es posible realizar distintas clasificaciones de ficheros en función de determinadas características. En nuestro caso nos vamos a centrar en las clasificaciones por el tipo de contenido y por el modo de acceso.

### 5.1. Contenido

Principalmente podemos distinguir dos tipos de ficheros en función de su contenido:

#### *5.1.1. Ficheros de texto*

Los datos se guardan en los ficheros en forma de texto, es decir, como secuencias de caracteres. El contenido de este tipo de ficheros puede ser leído por una persona.

#### *5.1.2. Ficheros binarios*

Los datos son guardados en su representación binaria, es decir, en secuencias de bytes. El contenido de este tipo de ficheros no puede ser leído por una persona.

### 5.2. Modo de acceso

Según el modo de acceso a los datos, los ficheros se pueden clasificar en:

#### *5.2.1. Ficheros secuenciales*

Se trata de archivos en los que el contenido se lee o escribe de forma continua. No se accede a un punto concreto del archivo. Para leer cualquier información necesitamos leer todos los datos desde el principio del fichero hasta llegar a la información deseada. En general los archivos de texto se suelen utilizar de forma secuencial.

### 5.2.2. Fichero de acceso directo

Se puede acceder a cualquier dato del archivo conociendo la posición en el mismo. Dicha posición se suele indicar en bytes.

## 6. APERTURA Y CIERRE DE FICHEROS

Para manejar ficheros vamos a hacer uso de la clase `java.io.File`.

### 6.1. Creación de ficheros

Para crear un fichero usaremos el constructor de la clase `File`, proporcionándole el nombre del fichero que queremos crear, y después ejecutaremos el método `createNewFile()`.

```
public File(String pathname)
```

```
public boolean createNewFile() throws IOException
```

Como podemos observar, el método `createNewFile` propaga excepciones `IOException`, por tanto, cuando trabajemos con ficheros en Java, siempre vamos a necesitar encapsularlo dentro de un `try...catch`

Ejemplo:

```
File archivo = new File("archivo.txt");  
archivo.createNewFile();
```

Esto nos creará el fichero “archivo.txt” en la misma ruta que se encuentra el proyecto.

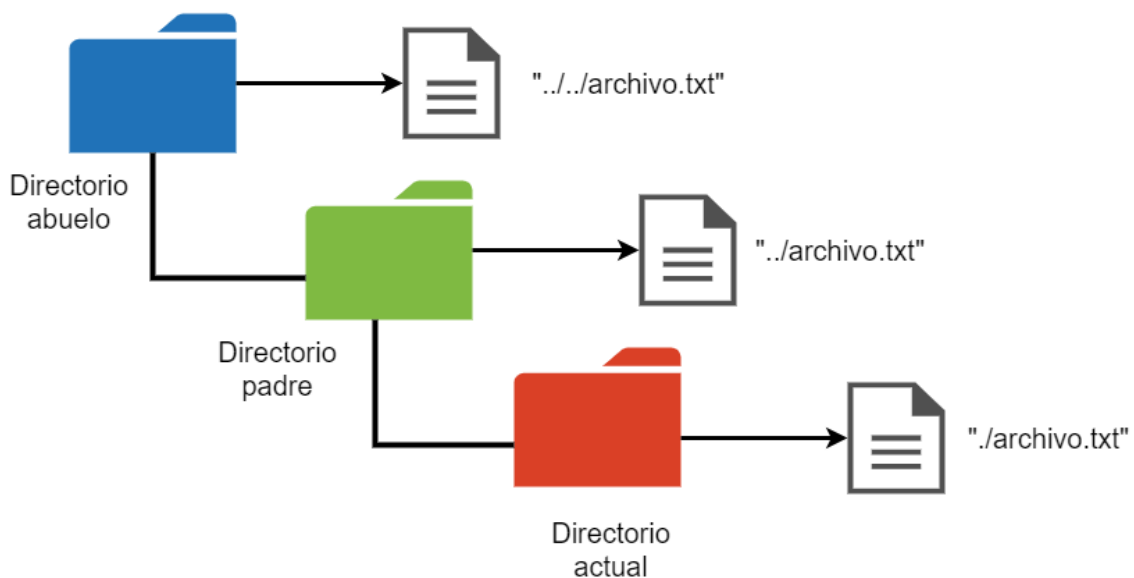
En el caso de queramos crear el fichero en una ruta específica, debemos dar la o la ruta absoluta o la ruta relativa del mismo. Por supuesto, la ruta de directorios deberá existir, de lo contrario nos dará error.

Ejemplo con ruta relativa:

```
File archivo01 = new File("./ruta01/archivo01.txt");
archivo01.createNewFile();
```

Cuando utilicemos rutas relativas debemos tener en cuenta varias cosas:

- En las rutas relativas se toma como referencia el directorio actual, es decir, el del proyecto.
- Si utilizamos “./” al comienzo de la ruta estamos indicando que comience desde el directorio actual.
- Si utilizamos “../” al comienzo de la ruta estamos indicando que comience desde el directorio padre del directorio actual. Podremos unir varios “../” para subir niveles de directorios, por ejemplo, con “../..” nos posicionaremos en directorio padre del directorio padre del directorio actual.
- Para separar niveles de directorios utilizamos el carácter “/”, por ejemplo, “../..ruta03/archivo.txt”

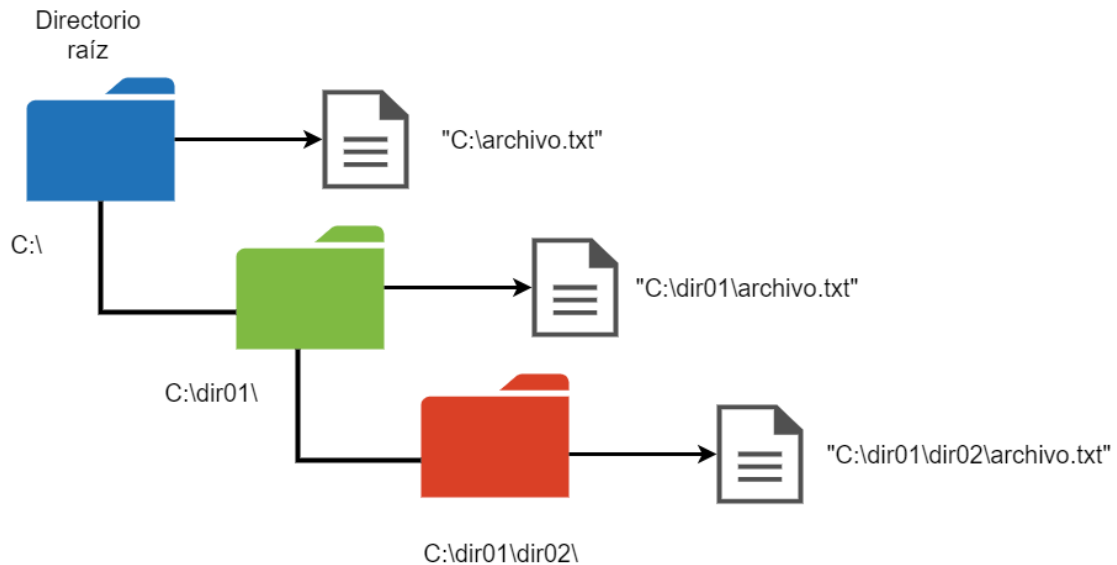


Ejemplo con ruta absoluta:

```
File archivo02 = new File("c:\\ruta02\\archivo02.txt");
```

```
archivo02.createNewFile();
```

Cuando utilizemos rutas absolutas debemos escribir toda la ruta completa, desde el directorio raíz. Para dividir los niveles de directorios debemos utilizar "\\".



El código completo quedaría de la siguiente manera:

```
public static void main(String[] args) {  
    try {  
        File archivo01 = new File("./ruta01/archivo01.txt");  
        if(archivo01.createNewFile())  
            System.out.println("El fichero se creó");  
        else  
            System.out.println("El fichero no se creó");  
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
}
```

## 6.2. Creación de directorios

Para crear directorios utilizaremos los métodos `mkdir()` y `makedirs()`.

```
public boolean mkdir()
```

```
public boolean mkdirs()
```

La diferencia entre `mkdir` y `mkdirs` consiste en que `mkdirs` creará también los directorios padre si no existen, mientras que `mkdir` solo crea el directorio final y si los directorios padres no existen no lo creará.

A la hora de crear los directorios debemos tener en cuenta los mismos factores que con la creación de ficheros, ya sean con ruta relativa o con ruta absoluta.

Veamos un ejemplo:

```
public static void main(String[] args) {
    try {

        File directorio01 = new File("./directorio01");
        if(directorio01.mkdir())
            System.out.println("Se ha creado el directorio01");
        else
            System.out.println("No se ha creado el directorio01");

        File directorio02 = new File("C:\\directorioPadre\\
\\directorio02");
        if(directorio02.mkdirs())
            System.out.println("Se ha creado el directorio02");
        else
            System.out.println("No se ha creado el directorio02");

    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
```

### 6.3. Eliminación de ficheros y directorios

Si lo que necesitamos es eliminar ficheros directorios haremos uso del método `delete()`.

```
public boolean delete()
```

Previamente crearemos un objeto de la clase File con un archivo o directorio existente y después ejecutaremos delete().

Cuando eliminemos un directorio, también eliminaremos todo su contenido.

```
public static void main(String[] args) {
    try {

        File archivo02 = new File("c:\\ruta02\\archivo02.txt");
        if(archivo02.delete())
            System.out.println("archivo02 fue eliminado");
        else
            System.out.println("archivo02 fue eliminado");

        File ruta01 = new File("./ruta01/");
        if(ruta01.delete())
            System.out.println("El directorio rut01 fue eliminado");
        else
            System.out.println("El directorio rut01 fue eliminado");

    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
```

#### 6.4. Renombrar o mover ficheros y directorios

Para renombrar o mover ficheros y directorios vamos a utilizar el mismo método:

```
public boolean renameTo(File dest)
```

En el parámetro introduciremos un objeto File con la nueva ruta y/o nombre.

```
public static void main(String[] args) {
    try {
```



```
File archivo01 = new File("./ruta01/archivo01.txt");
File archivo01renamed = new
File("./ruta01/archivo01renamed.txt");
if(archivo01.renameTo(archivo01renamed))
    System.out.println("El fichero se renombró");
else
    System.out.println("El fichero no se renombró");

File archivo02 = new File("./ruta01/archivo02.txt");
archivo02.createNewFile();
File archivo02moved = new File("./archivo02.txt");
if(archivo02.renameTo(archivo02moved))
    System.out.println("El fichero se movió");
else
    System.out.println("El fichero se movió");

File ruta01 = new File("./ruta01/");
File directorio01 = new File("./directorio01/ruta01/");
if(ruta01.renameTo(directorio01))
    System.out.println("El directorio fue movido");
else
    System.out.println("El directorio fue movido");

} catch (Exception e) {
    System.out.println(e.toString());
}

}
```

### 6.5. Leer contenido de directorios

Para conocer el contenido de un directorio tenemos dos opciones.

En primer lugar, podemos obtener los objetos File correspondientes a todos los archivos y directorios que se encuentran en el propio directorio con el método:

```
public File[] listFiles()
```

Este método nos devuelve un array de objetos File.

Si lo que nos interesa son solo los nombres, podemos utilizar:

```
public String[] list()
```

Veámoslo en código:

```
public static void main(String[] args) {  
    try {  
  
        File directorio = new File("./nbproject/");  
        File[] archivos = directorio.listFiles();  
        for(File archivo:archivos){  
            System.out.println(archivo.getName());  
        }  
  
        System.out.println("");  
  
        String[] archivosString = directorio.list();  
        for(String archString: archivosString){  
            System.out.println(archString);  
        }  
  
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
}
```

Para saber si un objeto File es una archivo o una carpeta, contamos con dos funciones propias de la clase:

- `public boolean isFile()`

- `public boolean isDirectory()`

Link de la ayuda oficial de java de la clase File:

<https://docs.oracle.com/javase/8/docs/api/java/io/File.html>

Practiquemos con la gestión de ficheros y directorios.

